

NAME: Fahad khan

ROLL NO: 23-SE-64

QUESTION NO -1

(1)

Zhang, Y., Liu, H., & Chen, X. (2024). An improved search algorithm for Efficient path planning in autonomous vehicles. IEEE transactions on intelligent transportation systems, 25(3), 1452–1465.

(2)

The research focuses on the problem of path planning for autonomous vehicles. In real-world environments, vehicles must find the shortest and safest path from a starting point to a destination while avoiding obstacles such as buildings, pedestrians, and other vehicles.

Traditional path planning methods often face issues such as:

High computational cost ,

Slow response in dynamic environments ,

Suboptimal path selection.

The paper aims to improve path planning efficiency so that autonomous vehicles can make fast and accurate decisions in real time environments.

(3)

The researcher used informed search algorithm specifically an improved version of A* algorithm.

(4)

The A* algorithm was applied to represent the environment as a grid-based map, where:

Each cell represents a possible position of the vehicle

Obstacles are marked as blocked cells

The start and goal positions are defined

The algorithm works using the function:

$$f(n)=g(n)+h(n)$$

Where:

$g(n)$ = actual cost from start node to current node

$h(n)$ = heuristic estimate from current node to goal

Modifications Made in the Paper

The researchers improved the standard A* algorithm by:

Enhanced Heuristic Function :

They used a more accurate heuristic to better estimate distance, reducing unnecessary exploration.

Dynamic Weight Adjustment :

The importance of heuristic vs actual cost was adjusted dynamically depending on the environment.

Obstacle Avoidance Optimization :

Additional checks were added to avoid unsafe or narrow paths.

Process Flow

1. Initialize start node
2. Add it to the open list
3. Expand nodes based on lowest $f(n)$.
4. Continuously update paths
5. Stop when goal node is reached

This allowed the system to find an optimal path efficiently.

(5)

The A* algorithm was suitable for this problem because:

Optimality: It guarantees the shortest path if the heuristic is admissible

Efficiency: It reduces unnecessary exploration compared to uninformed search (like BFS or DFS)

Flexibility: It can be modified with heuristics for different environments

Real-time capability: Faster than many traditional approaches when optimized

In autonomous driving, decisions must be made quickly and accurately, which makes A* a strong choice.

(6)

The improved A* algorithm showed significant improvements over traditional methods:

- Reduced computation time by approximately 25 - 30%
- Shorter path length compared to standard A*.
- Better obstacle avoidance, especially in complex environments..
- Improved real-time performance, making it suitable for autonomous systems.

The experiments demonstrated that the proposed method is more efficient and reliable, particularly in dynamic and complex road scenarios.

QUESTION NO – 2

Implementation OF MINIMAX AN ADVERSIAL SEARCHED ALGORITHM:

```
import random

# Node class for Binary Tree
class Node:
    def __init__(self, value=None):
        self.value = value
        self.left = None
        self.right = None

# Function to create a binary tree of given depth
def create_tree(depth):
    if depth == 0:
        # Leaf node with random value (positive or negative)
        return Node(random.randint(-10, 10))

    node = Node()
    node.left = create_tree(depth - 1)
    node.right = create_tree(depth - 1)
    return node
```

```

# Minimax Algorithm
def minimax(node, depth, is_maximizing, path, explored_nodes):

    # If leaf node
    if node.left is None and node.right is None:
        explored_nodes.append(node.value)
        return node.value, path

    if is_maximizing:
        max_eval = float('-inf')
        best_path = []

        # Explore left
        eval_left, path_left = minimax(node.left, depth + 1, False, path +
["L"], explored_nodes)

        # Explore right
        eval_right, path_right = minimax(node.right, depth + 1, False,
path + ["R"], explored_nodes)

        # Choose best
        if eval_left > eval_right:
            max_eval = eval_left
            best_path = path_left
        else:
            max_eval = eval_right
            best_path = path_right

        return max_eval, best_path

    else:
        min_eval = float('inf')
        best_path = []

        # Explore left
        eval_left, path_left = minimax(node.left, depth + 1, True, path +
["L"], explored_nodes)

        # Explore right

```

```

        eval_right, path_right = minimax(node.right, depth + 1, True, path
+ ["R"], explored_nodes)

# Choose best
if eval_left < eval_right:
    min_eval = eval_left
    best_path = path_left
else:
    min_eval = eval_right
    best_path = path_right

return min_eval, best_path

# MAIN PROGRAM
if __name__ == "__main__":
    depth = int(input("Enter tree depth (<= 5 recommended for demo): "))

    root = create_tree(depth)

    explored_nodes = []

    final_value, best_path = minimax(root, 0, True, [], explored_nodes)

    print("\n--- OUTPUT ---")
    print("Nodes explored (leaf values):", explored_nodes)
    print("Best Path (Root to Leaf):", " -> ".join(best_path))
    print("Final Evaluation Value:", final_value).

```

OUTPUT:

```

1  import random
2
3  # Node class for Binary Tree
4  class Node:
5      def __init__(self, value=None):
6          self.value = value
7          self.left = None
8          self.right = None
9
10 # Function to create a binary tree of given depth
11 def create_tree(depth):
12     if depth == 0:
13         # Leaf node with random value (positive or negative)
14         return Node(random.randint(-10, 10))
15
16     node = Node()
17     node.left = create_tree(depth - 1)
18     node.right = create_tree(depth - 1)
19     return node
20
21 # Minimax Algorithm
22 def minimax(node, depth, is_maximizing, path, explored_nodes):
23
24     # If Leaf node
25     if node.left is None and node.right is None:
26         explored_nodes.append(node.value)
27         return node.value, path
28
29     if is_maximizing:
30         max_eval = float('-inf')
31         best_path = []
32
33         # Explore left
34         eval_left, path_left = minimax(node.left, depth + 1, False, path + ["L"], explored_nodes)
35
36         # Explore right
37         eval_right, path_right = minimax(node.right, depth + 1, False, path + ["R"], explored_nodes)
38
39         # Choose best
40         if eval_left > eval_right:
41             max_eval = eval_left
42             best_path = path_left
43         else:
44             max_eval = eval_right
45             best_path = path_right
46
47     return max_eval, best_path
48
49     else:
50         min_eval = float('inf')
51         best_path = []
52
53         # Explore left
54         eval_left, path_left = minimax(node.left, depth + 1, True, path + ["L"], explored_nodes)
55
56         # Explore right
57         eval_right, path_right = minimax(node.right, depth + 1, True, path + ["R"], ex
58
59         # Choose best

```

Restart Visual Studio Code to apply the latest update.

Update Now

Later

Release Notes

```
    # Choose best
    if eval_left < eval_right:
        min_eval = eval_left
        best_path = path_left
    else:
        min_eval = eval_right
        best_path = path_right

    return min_eval, best_path
```

```
# MAIN PROGRAM
```

```
if __name__ == "__main__":
    depth = int(input("Enter tree depth (<= 5 recommended for demo): "))

    root = create_tree(depth)

    explored_nodes = []

    final_value, best_path = minimax(root, 0, True, [], explored_nodes)

    print("\n--- OUTPUT ---")
    print("Nodes explored (leaf values):", explored_nodes)
    print("Best Path (Root to Leaf):", " -> ".join(best_path))
    print("Final Evaluation Value:", final_value)
```

AI-quiz-1.py > ...

```
1  import random
2
3  # Node class for Binary Tree
4  class Node:
5      def __init__(self, value=None):
6          self.value = value
7          self.left = None
8          self.right = None
9
10 # Function to create a binary tree of given depth
11 def create_tree(depth):
12     if depth == 0:
13         # Leaf node with random value (positive or negative)
14         return Node(random.randint(-10, 10))
15
16     node = Node()
17     node.left = create_tree(depth - 1)
18     node.right = create_tree(depth - 1)
19     return node
20
21 # Minimax Algorithm
22 def minimax(node, depth, is_maximizing, path, explored_nodes):
23
24     # If Leaf node
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

powershell

● PS F:\DSA Python> & "C:\Users\Crown Tech\AppData\Local\Microsoft\WindowsApps\python3.12.exe" "f:/DSA Python/AI-quiz-1.py"
Enter tree depth (<= 5 recommended for demo): 4

--- OUTPUT ---

Nodes explored (leaf values): [9, 10, -10, 9, 6, 7, 3, 9, -6, 6, -9, -9, 5, 8, 8, -2]

Best Path (Root to Leaf): L -> R -> L -> L

Final Evaluation Value: 6

○ PS F:\DSA Python> █

Spaces: 4 UTF-8 () Python Finish Setup